Correlation-aware covariate selection and marginal-likelihood population updates for VAE-NLME

Two extensions to one-run variational-autoencoder estimation

Matthew L. Fidler

2026-08-19

Rohleff et al. (2025) showed that a variational autoencoder can estimate a nonlinear mixed-effects model and select its covariate model in a single training run, by minimizing a BICc-ELBO loss whose L0 term prices covariate effects directly. Two assumptions in that construction limit it on real covariate tables. First, the covariate selection scores each individual parameter independently and treats each candidate covariate as though no other candidate carried the same information; in pharmacometric data neither holds, and the consequence is not a noisy answer but a confidently wrong one that changes between iterations. Second, the population parameters are updated against the variational bound, which can only reach parameters that occupy the latent space – a structural parameter with no random effect is outside the method entirely. We describe extensions addressing both. For the first, covariates that are near-interchangeable are grouped into colinearity clusters, a selection is not displaced within a cluster without a margin measured in units of the selection penalty, near-ties are reported rather than silently resolved, and – when the model declares correlated random effects – a joint pass reconsiders which parameter a covariate belongs to. We show that this last pass is provably vacuous under a diagonal random-effect covariance, which is why it is gated on the model declaring the correlation rather than attempted always. For the second, the population M-step optimizes the full outer (Laplace) objective and is stepped by the exact analytic outer gradient, so a non-mu-referenced structural parameter is estimated by differentiating the same functional the fit reports rather than by a derivative-free search on a different one. We also describe the exact branch-and-bound that replaced the reference implementation's mixed-integer quadratic program, and the role `L0Learn` plays as a proposal generator whose own objective is never allowed to decide a selection.

Introduction

Rohleff, Kloft, and Huisinga (2025) introduced a variational autoencoder for nonlinear mixed-effects (NLME) estimation in which an amortized LSTM encoder learns each subject's posterior over individual parameters directly from that subject's observations, and an ODE decoder reconstructs the observations from the sampled latent variables. Training maximizes an evidence lower bound (ELBO) on the marginal likelihood. Their key observation is that covariate selection need not be a separate outer loop over candidate models: adding an L0 penalty on the covariate effect matrix turns the ELBO into a **BICc-ELBO**, and one training run then delivers both the population parameters and the covariate model. On two case studies this reproduced the covariate model chosen by SCM and COSSAC at a fraction of the cost.

That construction is attractive precisely because it collapses a search that is conventionally combinatorial into a single optimization. It also inherits two assumptions from that collapse, and this paper is about what happens when they fail.

The covariate selection scores parameters independently and covariates atomically. The L0 problem is solved over the full covariate effect matrix $\beta \in \mathbb{R}^{n_z \times n_c}$, but the objective is a sum of independent per-parameter least-squares fits, and a covariate enters or does not on its own merits. Real covariate tables are not like this. Weight, lean body mass, body surface area and BMI measure much the same thing; clearance and volume are routinely correlated. Under either kind of correlation the method does not fail loudly – it returns a specific, plausible covariate model that happens to be one of several the data cannot distinguish, and which one it returns changes from iteration to iteration.

The population parameters are updated against the bound. Equation (10) of Rohleff, Kloft, and Huisinga (2025) updates $\theta = (z_{\text{pop}}, \beta, \Omega, a)$ by maximizing $\mathcal{L}_\psi(x; \theta)$. This is well defined for any parameter that appears in the latent prior. A structural parameter with **no random effect** does not: it is not part of z, so the bound is flat in it and the M-step cannot move it. In the reference implementation such a parameter is simply not estimated.

We describe extensions addressing both, as implemented in `nlmixr2est` (Fidler et al. 2026). Section 2 recaps the method. Section 3 covers the covariate extensions, and includes a separability result that determines when one of them can do anything at all. Section 4 covers the outer-gradient population update. Section 5 reports the benchmark.

Background

The bound and the BICc-ELBO

For N subjects with observations x_i, latent individual parameters z_i, encoder $q_\psi(z_i \mid x_i) = \mathcal{N}(\mu_i, \Sigma_i)$ and prior $z_i = z_{\text{pop}} + \beta c_i + \eta_i$, $\eta_i \sim \mathcal{N}(0, \Omega)$, the ELBO is

$$\mathcal{L}_\psi(x;\theta) = \sum_{i=1}^{N} \mathbb{E}_{z_i \sim q_\psi} \left[\log p(x_i \mid z_i)\right] - \sum_{i=1}^{N} D_{\mathrm{KL}}\big(q_\psi(z_i \mid x_i) \,\|\, p(z_i)\big)\,. \tag{1}$$

Rohleff, Kloft, and Huisinga (2025) add an L0 penalty on the covariate effect matrix, giving the BICc-ELBO

$$\mathcal{L}_\psi^{\mathrm{BICc}}(x,\theta) = -2\mathcal{L}_\psi(x;\theta) + \log(N)\,\|\beta\|_0 + C, \tag{2}$$

and solve for $(z_{\mathrm{pop}}, \beta)$ by minimizing Equation 2 while Ω and a follow the ordinary ELBO update. Because the bound is quadratic in $(z_{\mathrm{pop}}, \beta)$, the inner problem is a mixed-integer quadratic program (MIQP).

The selection subproblem

Written per latent dimension k, with y_k the encoder's posterior means for that dimension and S_k the support (the set of covariate columns entering k), the M-step minimizes

$$\mathrm{score}(S_k) = \frac{\mathrm{RSS}_k(S_k)}{\omega_k} + \lambda|S_k|, \qquad \lambda = \log N, \tag{3}$$

where RSS_k is the residual sum of squares of the ordinary least-squares fit of y_k on an intercept plus the columns of S_k. Two properties of Equation 3 matter throughout and are easy to overlook.

The **sample size is the number of subjects**, not observations. A rich study with thousands of concentration measurements still offers only tens of rows to this regression.

The **response is an estimate, not data**. y_k is the encoder's posterior mean, refreshed every training iteration. It carries encoder noise and it moves.

Extension 1: covariate selection under correlation

From MIQP to an exact branch-and-bound

The reference implementation solves Equation 2 with `cvxpy` and a commercial MIQP solver. `nlmixr2est` instead solves Equation 3 directly by branch-and-bound, with the "fit all remaining columns" residual bound; the prune is admissible, so the result is the exact minimizer, and no commercial dependency is required. Past roughly 2^{17} feasible supports the node count becomes impractical, and the search switches to having `L0Learn` (Hazimeh and Mazumder 2020) propose supports.

The design of that fallback is worth stating, because it is what keeps the approximate path from changing answers on its own. **`L0Learn` proposes; it never decides.** Every proposed support is re-scored with Equation 3 – same OLS, same penalty, same tie-break – and then polished by an add/drop/swap local search under the same objective. The proposer's internal scaling, its λ grid and its γ grid therefore cannot shift a selection; they can only change which subsets are looked at. Two consequences follow. Both the `L0` and `L0L2` penalty paths are pooled, and every support from every γ is harvested, because extra candidates are free of risk and can only improve the answer. Cross-validating γ, which would be the natural instinct, is strictly worse than this: it selects one γ where the pooled path already takes all of them. We also verified that the proposals are invariant to column scaling (`L0Learn` normalizes internally), so the usual advice to standardize before an L2-penalized fit does not apply here.

`nlmixr2est` also searches a richer model space than a plain β matrix. A continuous covariate contributes one column per functional **shape** it may take (allometric, linear, log, centered, and a two-armed hockey stick knotted at the centering value); shapes of one covariate share a **mutual-exclusion group** so at most one may enter a parameter; and the two arms of a hockey stick form an **atomic block** selected all-or-none. The unit of search is therefore the block, not the column, and feasibility is checked at every leaf.

Why exact optimization does not solve colinearity

The natural diagnosis of a false selection between two correlated covariates is that the search failed to consider the alternative. On this implementation that diagnosis is simply false: the branch-and-bound enumerates every feasible support, and the `L0Learn` path runs add/drop/swap to a local optimum. Swapping weight for lean body mass is a move both engines score.

The difficulty is that Equation 3 is a **noisy criterion**. With n equal to the number of subjects and y_k an estimate that moves each iteration, two columns correlated at 0.98 differ in fit by less than the iteration-to-iteration movement of the response. The exact minimizer of a noisy criterion is still arbitrary within the noise, and it changes its mind. Optimizing harder cannot fix this, and neither can a better proposer: the problem is the objective's resolution, not the search's reach.

This reframing is the substantive point of this section. It implies the remedy must act on **stability and reporting**, not on search quality.

Colinearity clusters

Covariates whose absolute correlation reaches a threshold ρ_c are grouped into a colinearity cluster. Clustering is done at the level of the *covariate* (the mutual-exclusion group), not the search column, and only cross-group column pairs are ever compared. Two consequences are structural rather than special-cased: alternate shapes of one covariate can never cluster with

each other, and neither can the arms of one hockey block, since a block never spans groups. The resulting partition is always a coarsening of the mutual-exclusion groups, which is the invariant the rest of the machinery relies on.

Groups are joined by single linkage. At a high threshold, chaining is rare, and because a cluster is a *label* rather than a constraint – it never forbids a selection and never changes Equation 3 – an over-generous cluster costs a few extra evaluations, never an unreachable model.

Stability: hysteresis within a cluster

Within a cluster, the covariate selected on the previous iteration is not displaced by a mate unless that mate improves Equation 3 by more than $\tau_h \lambda$: a fraction of **one covariate's selection penalty**.

The unit matters. At the optimum RSS_k/ω_k is of order N while the decision scale is $\lambda = \log N$, so a margin expressed as a percentage of the objective would mean something different in every study – roughly 0.2 penalties at $N = 100$ and 1.5 at $N = 1000$. Anchoring to λ makes the knob invariant to sample size, to ω and to design scale, and gives it a reading: *a challenger must win by a quarter of one covariate's BIC cost.*

Hysteresis is applied to the support the search has already returned, never inside the scoring function. This is deliberate: a history-dependent term inside the leaf would falsify the branch-and-bound's admissible bound and destroy the exactness the rest of the method depends on. The search remains an exact minimizer of Equation 3; only its output is stabilized.

Reporting near-ties

Stability is not correctness. When the data cannot distinguish two covariates, what a modeler needs is to be told so. Cluster mates scoring within $\tau_a \lambda$ of the selection are recorded on the fit, with the parameter, the selected covariate, the mate, and the score gap.

Alternates are restricted to mates from a *different* covariate. Alternate shapes of one covariate are near-tied on almost every model – an allometric and a linear weight effect span nearly the same fit – so admitting them would make the report fire universally and carry no information.

Centering does not change the correlations – except where it does

Continuous covariates are centered on a population value before entering the search, and it is worth being precise about what that does to the correlations the clustering sees, because the intuition cuts both ways.

For every **affine** shape the answer is nothing at all. Correlation is invariant to separate affine transformations of each variable, and lin = COV − ctr, center = COV/ctr and identity = COV

are affine in the covariate, while power $= \log(\mathrm{COV}/\mathrm{ctr}) = \log \mathrm{COV} - \log \mathrm{ctr}$ is affine in $\log \mathrm{COV}$. Measured on the simulated pair below, the linear columns reproduce the raw correlation and the allometric columns reproduce the log-scale correlation, to every digit:

quantity	correlation
raw $\mathrm{cor}(\mathrm{WT}, \mathrm{LBM})$	0.989939
`lin` columns	0.989939
$\mathrm{cor}(\log \mathrm{WT}, \log \mathrm{LBM})$	0.989102
`power` columns	0.989102

So a `covCenter` override cannot move an affine shape into or out of a cluster. It also means the *shape* matters slightly even when the centering does not: the allometric and linear columns of the same covariate pair differ in correlation, here in the third decimal, and more for a skewed covariate.

The **hockey stick is the exception, and the exception is structural**: its knot *is* the centering value. Moving the center does not shift the column, it redefines it, so its correlation with anything else genuinely changes. Moving the centers from the medians to conventional values in the same data:

shape	at the median	at `WT = 70`, `LBM = 55`
`power`	0.989102	0.989102
`lin`	0.989939	0.989939
`hockeyLow`	0.984072	**0.961063**
`hockeyHi`	0.978921	**0.986965**

Because a cluster is formed from the **maximum** correlation over cross-group column pairs, a `covCenter` override can therefore change whether two covariates cluster, and it can do so only through the hockey columns. This is worth stating plainly for two reasons: it is a real dependence of the covariate model on a setting usually thought of as cosmetic, and it means the clusters must be computed after any centering has been resolved – which is where the implementation computes them.

Correlated parameters: the cross-parameter refinement

The second failure mode is structurally different. Each latent dimension runs its own search, and the coupling between dimensions enters only through a Gauss-Seidel offset built from the *previous* iteration's residuals, which is held fixed while a dimension searches. No per-dimension search can therefore observe that a covariate assigned to clearance would be better explained on volume.

When the model declares correlated random effects, a further pass makes that comparison. Dimensions are grouped by the empirical correlation of their posterior means, with **sticky membership** – a pair joins at ρ_ϕ^+ and leaves only below ρ_ϕ^- – so a correlation wandering near the threshold does not reshuffle the grouping every iteration. Groups are connected components, hence a partition of the dimensions. Over each group the pass considers moving a selected block to another dimension, adding it there, or dropping it, and scores the whole group jointly.

The joint objective

The obvious joint score – the sum of the per-dimension Gauss-Seidel scores over the group – is wrong, and wrong precisely where it matters. Each dimension's surrogate contains the cross term $2P_{kl}r_k^\top r_l$ for every partner l; summing over k counts each within-group cross term **twice**. Those cross terms are exactly what a move changes, so the surrogate is biased on the comparisons the pass exists to make.

Instead the group's stationarity conditions are solved directly. With $Z_k = [\mathbf{1} \mid X_{S_k}]$, $P = \Omega^{-1}$, and $u_k = \sum_{j \notin G} P_{kj} r_j$ the contribution of the dimensions outside the group,

$$\sum_{l \in G} P_{kl}\, Z_k^\top Z_l\, \theta_l = Z_k^\top \left(\sum_{l \in G} P_{kl}\, y_l + u_k \right), \qquad k \in G, \tag{4}$$

one dense linear system of size $|G| + \sum_k |S_k|$. Two properties are worth recording. Equation 4 is the **fixed point** of the Gauss-Seidel offset recursion, so it is what "recompute the offsets inside the move evaluation" converges to, obtained in closed form with no sweep count to tune. And when P is diagonal it reduces exactly to $\sum_k \mathrm{RSS}_k/\omega_k + \lambda|S|$.

When the refinement can do nothing

That last property has a consequence that determines the whole design.

> **Proposition.** If Ω is diagonal, no cross-dimension move can strictly improve the joint objective.

Proof sketch. With Ω diagonal the offsets vanish and $P_{kk} = 1/\omega_k$, so the joint objective is $\sum_k [\mathrm{RSS}_k(S_k)/\omega_k + \lambda|S_k|]$ – a sum of terms each depending on exactly one S_k. The minimum of a separable sum is the sum of the per-term minima, and each per-dimension search already returns its exact minimizer. □

So under a diagonal Ω the pass is not merely usually unhelpful; it cannot help. The implementation therefore gates it on the model declaring correlated random effects, and when correlated dimensions are detected under a diagonal Ω it says so rather than doing the work and reporting nothing.

We note what this does *not* claim. Mis-attribution between correlated parameters under a declared-diagonal Ω is real; it is simply invisible to the M-step, which conditions on frozen posterior means. The correlation there lives in the encoder. The honest remedy is to declare the correlation, and the implementation says so.

What these mechanisms do not do

They make a near-arbitrary choice stable and visible. They do not make it correct. If two covariates are interchangeable in the data, no selection criterion can identify which is mechanistically real; that is a question for the science and, if it matters, for a design that separates them. What the fit can now report is that the question exists.

Extension 2: the outer gradient and the population parameters

What the bound cannot reach

The ELBO M-step of Equation 1 updates parameters that appear in the latent prior. A structural population parameter with **no random effect** – a non-mu-referenced θ – does not appear there, so the bound is flat in it.

The workaround of injecting a small random effect so the parameter enters the latent space is available and implemented, but it changes the model: it adds a variance component that was not in the analyst's model and must then be reported or discarded.

Optimizing the objective that is reported

`nlmixr2est` instead estimates such a parameter directly, in the M-step, against the **full outer objective**: the frozen-eta joint likelihood *plus* the Laplace determinant, $\frac{1}{2}\log|\Omega^{-1}|$, and the DV-transform Jacobian. This is a deliberate deviation from Rohleff, Kloft, and Huisinga (2025), whose M-step targets the plain variational bound.

The reason is not that the outer objective is a better bound – it is not a bound at all, it is the marginal (Laplace) objective the fit ultimately reports. Targeting it keeps the quantity being optimized equal to the quantity being reported. Under the reference behavior the two differ by terms depending on the non-mu parameters through the eta Hessian, so the two objectives can land on different estimates – and, because those estimates feed the latent means the covariate search regresses on, on different **covariate models**. The reference behavior remains available for reproduction.

Stepping it with the exact gradient

Two optimizers are provided for that M-step.

The default performs a bounded derivative-free (`bobyqa`) regression against the outer objective, honoring the `ini()` bounds, re-optimized each M-step and blended with the M-step gain.

The alternative steps the parameter with the **exact analytic outer gradient** – the Almquist sensitivity equations (Almquist, Leander, and Jirstrand 2015), the same machinery behind `nlmixr2est`'s fast FOCEi gradient. One augmented sensitivity solve per M-step replaces a full derivative-free sweep, in which each function evaluation costs an N-subject inner likelihood pass.

Both optimize the same functional, so this changes the optimizer, not the target. It is also the more natural fit for the method: the gradient is handed to the **same Adam machinery that moves the encoder weights**, so the population parameter is learned alongside the rest of the model on a shared schedule – same gain, same KL warm-up gate – rather than pausing each M-step to run a separate optimizer to convergence and adopting its answer. In this sense the extension restores the character of the original method, in which everything is learned in one run, to a parameter the bound could not reach.

The two therefore differ in **convergence behavior**, not merely in cost, and that difference is larger than the cost difference. The derivative-free path re-optimizes the parameter to convergence within every M-step, so it arrives near the optimum almost immediately and is essentially independent of the starting value. The gradient path takes a single Adam step per M-step on the shared schedule, so it must *traverse* the distance from the initial estimate to the optimum over the course of training.

Starting a structural parameter well away from its true value ($\log 20$ against a simulated $\log 31$) makes this explicit (three replicates, error against each replicate's own FOCEi maximum-likelihood value):

Table 3: Accuracy and cost against run length.

iterations	derivative-free |err|	analytic |err|	derivative-free (s)	analytic (s)
80	0.00108	0.29312	7.1	6.8
150	0.00160	0.01484	12.0	11.1
300	0.00156	0.00198	25.5	22.3
500	0.00115	0.00056	43.9	38.0

The two curves have different shapes, and that is the finding. The derivative-free path is at its accuracy floor from the shortest run and **stays there** – more iterations do not improve it, because it already re-optimizes to convergence inside every M-step. Its floor is set by what it

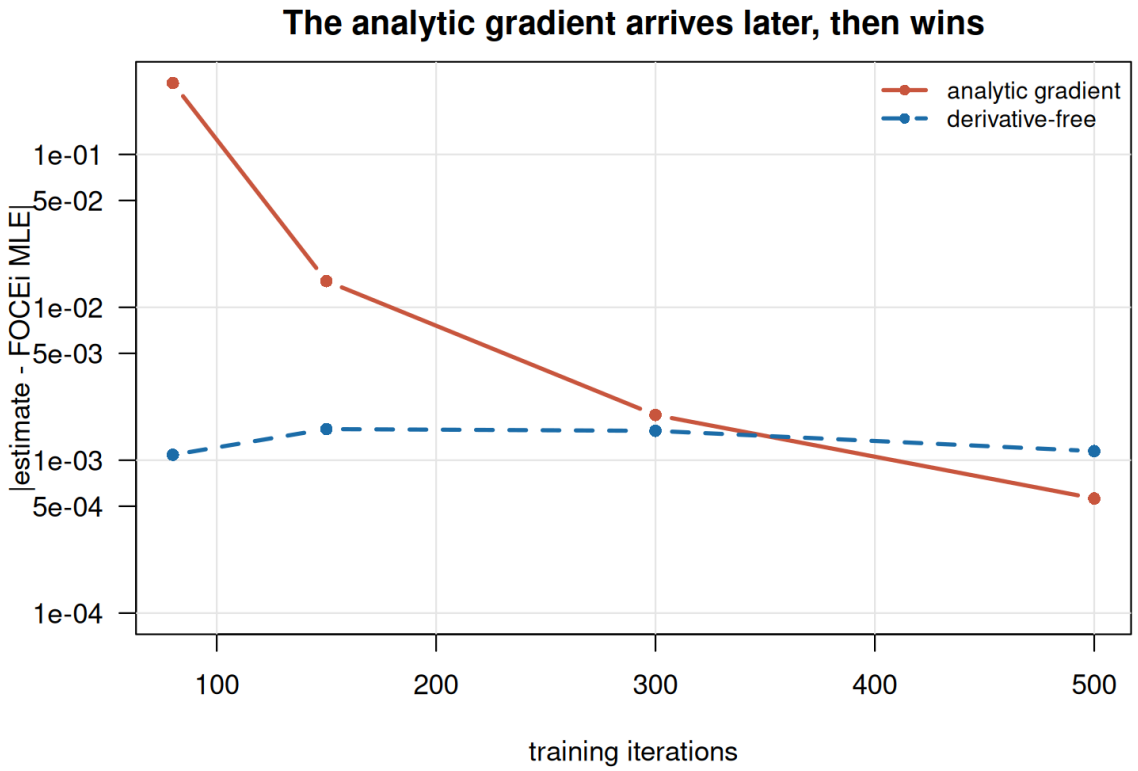

Figure 1: The derivative-free path is at its floor immediately; the analytic gradient has to get there, then goes past it.

optimizes, not by how long it runs. The analytic-gradient path starts far away – at 80 iterations it has covered only about two thirds of the distance from its starting value and is two orders of magnitude worse – but it converges monotonically, matches the derivative-free path by 300 iterations, and by 500 is roughly twice as accurate, having gone below that floor.

On cost, the analytic path was the faster of the two at **every** run length measured, by about 10%. This differs from the figures quoted in the implementation's own documentation, which report it as the slower option; we measured on an ODE model, where the analytic path genuinely applies, and note that on a solved-form (`linCmt()`) model it is out of analytic scope and silently falls back to the derivative-free optimizer, so a timing comparison there does not measure what it appears to.

This refines the claim that the analytic gradient is simply the more accurate option. It is more accurate **given enough iterations**, and it is cheaper per iteration; but it inherits the training schedule, so a structural parameter started far from its optimum needs enough iterations to arrive, and a run truncated before that reports an estimate still in transit. The derivative-free path has no such dependence and is the safer choice for a short run or a poor initial estimate. Run length and starting value, not cost, are the deciding consideration.

The analytic path applies to conditionally Gaussian models and to a single non-Gaussian endpoint. Outside that scope – `linCmt()`, IOV, multi-endpoint or censored general likelihoods – it falls back to the derivative-free optimizer with a note, rather than silently changing target.

Benchmark

Colinear recovery and reporting

Ten replicates per cell. `on` is the default configuration, `off` is `covSelectColinear = FALSE`, i.e. the previous behavior.

Table 4: Selection outcome and reporting, by decoy correlation.

rho	effect	arm	true covariate	decoy	nothing	alternates/fit	holds/fit
0.50	0.00	off	0.2	0.0	0.8	0.0	0.0
0.50	0.00	on	0.2	0.0	0.8	0.0	0.0
0.99	0.00	off	0.1	0.1	0.7	0.0	0.0
0.99	0.00	on	0.1	0.1	0.7	0.8	0.0
0.50	0.75	off	0.8	0.0	0.1	0.0	0.0
0.50	0.75	on	0.8	0.0	0.1	0.0	0.0
0.99	0.75	off	0.5	0.3	0.0	0.0	0.0
0.99	0.75	on	0.5	0.3	0.0	1.6	0.1

The selection is **identical** with the mechanisms on and off – the same fractions pick the true covariate, the decoy, or nothing. What differs is whether the modeller is told. At a decoy correlation of 0.99 with a real effect, the decoy is selected in a substantial minority of fits, and only the `on` arm reports alternates. At 0.50 neither arm reports anything, and the true covariate is recovered in most fits.

That is the intended behavior stated precisely: these mechanisms were never meant to change which covariate wins. They make an unreproducible choice visible.

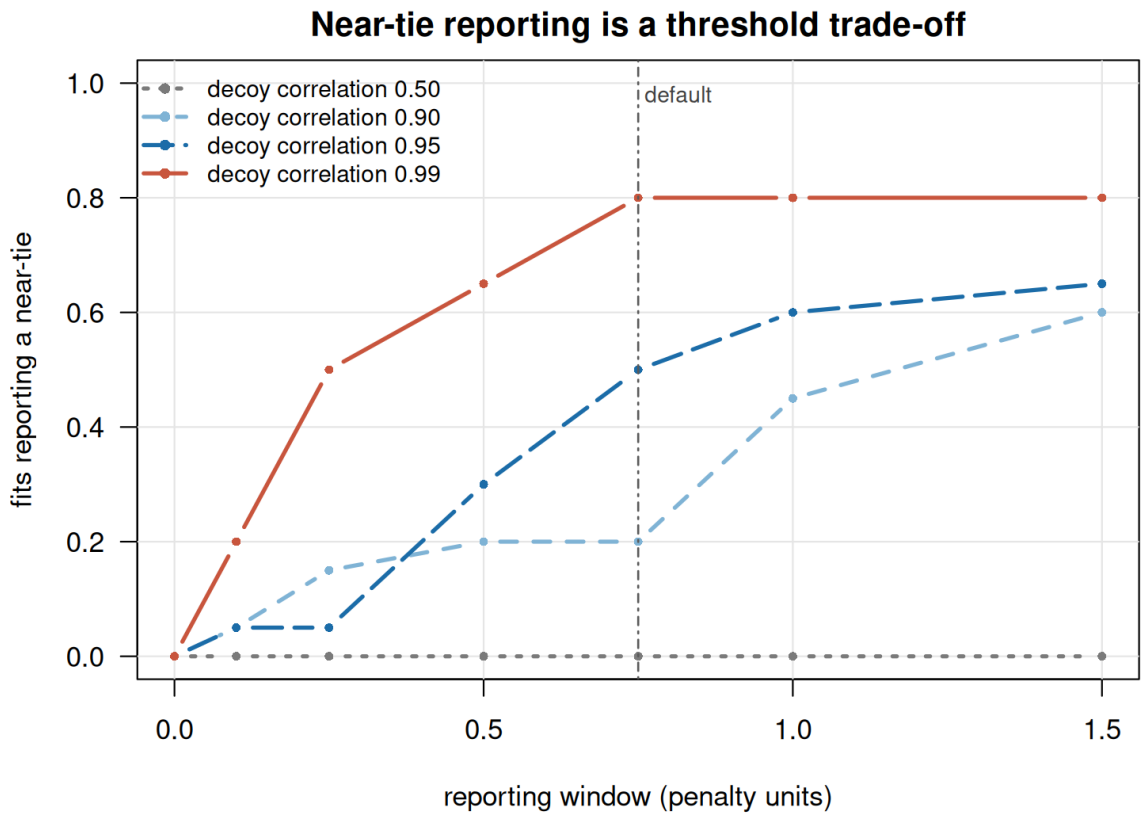

Figure 2: Reporting fires where the covariates are indistinguishable and stays flat where they are not.

What the two mechanisms actually contribute

The reporting and the hold were introduced together, and the benchmark separates them. They do not contribute equally.

The report fires on ambiguity, not on error

Conditioning on whether anything was selected at all:

ρ	true effect	nothing selected	selected, reported
0.50	0.75	1 fit, 0 reported	9 fits, **0%**
0.50	none	6 fits, 0 reported	4 fits, **0%**
0.99	0.75	0 fits	10 fits, **70%**
0.99	none	5 fits, 0 reported	5 fits, **60%**

Two properties, both wanted. The report never fires when nothing was selected – there is no choice to call ambiguous. And its rate tracks the **correlation**, not whether the selected effect is real: a spurious pick between two covariates correlated at 0.99 is reported at essentially the same rate as a genuine one. That is correct. The question the report answers is "could this data have distinguished these covariates?", which is a property of the design, and it is just as worth answering when the selected effect is noise.

The hold has almost nothing to act on

The hold can only fire when this iteration's winner displaced last iteration's within a cluster. Setting the margin so large that **every** eligible revert is accepted turns its counter into a count of opportunities. Across eight datasets at $\rho = 0.99$ with a real effect, that count is:

	opportunities per fit	maximum
all datasets	**0.12**	1

One dataset in eight offered a single opportunity; the other seven offered none. Refitting one dataset under twelve different training seeds gave zero opportunities and the *identical* covariate every time. The observed hold rate in the main benchmark (0.1 per fit) is therefore not a margin set too tight – it is very nearly the whole of what was available to take.

So on these designs the near-tie report is doing essentially all of the work, and the hold is close to inert. That is worth stating plainly rather than presenting the two as equal contributions.

Why: the response is noisy, but not in the way that was assumed

The motivation for the hold was iteration-to-iteration movement in the selection response. That movement is real – and, notably, the smoothing intended to damp it is inactive for most of a default run. The EMA gain is exactly 1 until iteration `gammaIter` (default 250 of 300), and

$s_1 \mathrel{+}= 1 \cdot (\mu - s_1)$ leaves s_1 **identical** to the raw posterior means. We confirmed this directly: `covSelectSmooth = TRUE` and `FALSE` give bit-identical fits at 150 iterations. Any run shorter than `gammaIter` is therefore selecting on unsmoothed posterior means whatever that setting says.

Yet the selection still does not flip within a run. The instability that the benchmark does show is **between datasets** – at $\rho = 0.99$ the true covariate wins in half the replicates and the decoy in a third – and no within-run mechanism can address that, because each run sees only its own data. This is the sense in which the near-arbitrariness is real but is not an optimization defect: every individual run is stable and reproducible, and the answers still disagree with one another across datasets.

Cross-parameter attribution

Table 7: The cross-parameter refinement, and its diagonal-omega control.

arm	true covariate on CL	anything on V	pairs found	moves scored	moves accepted	advisory raised
block	1	0.1	100	110	0.8	0
block-off	1	0.1	0	0	0.0	0
diag	1	0.0	100	0	0.0	1

The `diag` arm is the empirical form of the Proposition: the same correlated pairs are detected as in the `block` arm, and **not one move is scored**, because with a diagonal covariance there is provably nothing to gain. The advisory is raised in its place. With the feature off, no grouping is computed at all.

Calibrating the near-tie window

The reporting window `covSelectAltTol` is the one setting whose default cannot be argued from first principles, so it was measured. Twenty replicates of the colinear design were fitted at each of four decoy correlations, each fitted **once** with a deliberately wide window so that every within-cluster gap is recorded; every candidate threshold is then scored post-hoc from those gaps, so all candidates see identical fits.

The gap to the competing covariate, in penalty units, falls sharply with correlation ($n = 20$ per row):

decoy correlation	mean gap	sd	min	max
0.90	1.416	1.447	0.099	4.877
0.95	0.852	0.737	0.044	2.667
0.99	0.258	0.212	0.020	0.747

At $\rho = 0.99$ the mean gap is about a quarter of one covariate's BIC cost – the two covariates are, by the criterion's own scale, not distinguishable. At $\rho = 0.90$ it is 1.4 penalties, a real difference. That gradient is what a reporting threshold has to sit inside.

The fraction of fits that would report a near-tie, by threshold:

ρ	0.1	0.25	0.5	**0.75**	1.0	1.5
0.50	0.00	0.00	0.00	0.00	0.00	0.00
0.90	0.05	0.15	0.20	0.20	0.45	0.60
0.95	0.05	0.05	0.30	0.50	0.60	0.65
0.99	0.20	0.50	0.65	0.80	0.80	0.80

Three things follow.

The $\rho = 0.50$ control is **clean at every threshold**: a design whose covariates are genuinely separable never reports, even at 1.5 penalties. The report is not simply firing whenever a cluster exists.

An 0.1 window – the value first shipped – reports in only 20% of the $\rho = 0.99$ fits, where the selected covariate demonstrably changes between replicates. A mechanism that stays silent in four of five cases of exactly the situation it exists for is mis-calibrated, and this is what the benchmark was built to detect.

Between 0.5 and 0.75 the false-alarm rate at $\rho = 0.90$ **does not move** (0.20 in both), while sensitivity rises at $\rho = 0.95$ (0.30 to 0.50) and at $\rho = 0.99$ (0.65 to 0.80). At 1.0 that stops being true: the $\rho = 0.90$ rate more than doubles, to 0.45, and the report starts firing on pairs the data can separate. On this evidence 0.75 dominates 0.5 – equal specificity, better sensitivity – and 1.0 is where the trade-off turns. **0.75 is the default adopted.** With 20 replicates per cell the difference between 0.5 and 0.75 at $\rho = 0.95$ is 6 fits against 10, so the margin is suggestive rather than decisive; the defensible range is $[0.5, 0.75]$ and the evidence against 0.1 is what is unambiguous.

A caveat on the outcome shares. Across all correlations, 20% of fits selected **both** covariates on the same parameter, which is a consequence of the deliberate decision that a cluster labels rather than constrains. At $\rho = 0.99$ the decoy alone was selected in 35% of fits and the true covariate alone in 45%. Win counts therefore overlap and do not sum to one; the gap distribution above is the sounder statement of indistinguishability.

Benchmark design

The remaining benchmark answers four questions, each with a negative control so that a mechanism cannot appear to succeed by always firing.

1. **Colinear recovery.** With a true covariate and a decoy correlated with it at 0.99, how often is the true covariate the one reported, and how often does the selection change between iterations? Control: an uncorrelated design, where the mechanisms must be inert.
2. **Near-tie reporting calibration.** When the decoy is genuinely indistinguishable, is it reported? When the effect is strong and the decoy is distinguishable, is the report silent?
3. **Cross-parameter attribution.** With correlated parameters and the truth on one of them, does the joint pass move the covariate to the correct parameter? Control: the same model with a diagonal Ω, where by the Proposition nothing may move.
4. **Outer-gradient accuracy and cost.** For a non-mu-referenced structural parameter, the distance from the FOCEi maximum-likelihood value, and the wall-clock cost, for the derivative-free and analytic-gradient optimizers under both objectives.

Simulation archetypes are in `R/scenarios.R`; the driver is `R/run_benchmark.R`, which checkpoints one `.rds` per fit so a run can be interrupted and resumed.

Discussion

The two extensions are independent in implementation and related in motivation. Both concern the gap between what the variational bound conveniently supplies and what the model being fitted actually requires.

For covariate selection, the bound supplies a criterion whose resolution is set by the number of subjects and by the encoder's own noise. Treating that criterion as though it were sharp yields a definite answer that is not reproducible; treating it as blunt – stabilizing within the resolution and reporting what falls inside it – yields an answer that is, together with an honest statement of what the data could not decide.

For the population parameters, the bound supplies an objective that is convenient but is not the one reported, and which is flat in any parameter outside the latent space. Optimizing the reported objective, and differentiating it exactly, closes both gaps at once.

Limitations

The colinearity mechanisms are **defaults**, and defaults chosen from measurement on a specific set of designs. The thresholds are exposed and the whole path can be disabled to reproduce the previous behavior exactly.

The cross-parameter refinement requires a declared correlated Ω. This is a real restriction, not a tuning choice, and the Proposition says why.

The analytic outer gradient is restricted to models in analytic scope, and falls back with a note outside it.

Finally, none of this addresses identifiability. A cluster tells you two covariates are interchangeable **in this data set**; it does not tell you which is causal, and no amount of selection machinery can.

References

Almquist, Joachim, Jacob Leander, and Mats Jirstrand. 2015. "Using Sensitivity Equations for Computing Gradients of the FOCE and FOCEI Approximations to the Population Likelihood." *Journal of Pharmacokinetics and Pharmacodynamics* 42 (3): 191–209.

Fidler, Matthew L. et al. 2026. *Nlmixr2est: Nonlinear Mixed Effects Models in Population Pharmacokinetics and Pharmacodynamics*.

Hazimeh, Hussein, and Rahul Mazumder. 2020. "Fast Best Subset Selection: Coordinate Descent and Local Combinatorial Optimization Algorithms." *Operations Research* 68 (5): 1517–37.

Rohleff, Jan, Charlotte Kloft, and Wilhelm Huisinga. 2025. "Redefining Parameter Estimation and Covariate Selection via Variational Autoencoders: One Run Is All You Need." *CPT: Pharmacometrics & Systems Pharmacology* 14: 2233–45.